SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4304

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I P,Yaswanth Choudhary(192411136)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: **P,Yaswanth Choudhary**

1. Online Symposium Portal:

Given HTML

```
<form action="/register" method="POST">
   <input type="text" name="name">
   <input type="email" name="email">
   <button>Submit</button>
</form>
```

1.1 Appropriate HTML5 semantic elements missing

The form can be improved using semantic elements such as:

- <header> – for the page or symposium heading.
- <main> – for the main content of the page.
- <section> – for grouping the registration content.
- <footer> – for footer information.
- <label> – to provide accessible descriptions for form fields.
- <fieldset> and <legend> – to group related form controls.

1.2 Standards-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Online Symposium Registration</title>
</head>

<body>

<header>
   <h1>College Online Symposium</h1>
</header>

<main>
   <section>
      <h2>Student Registration</h2>

      <form action="/register" method="POST">

         <fieldset>
            <legend>Registration Details</legend>

            <label for="name">Name:</label>
            <input type="text" id="name" name="name" required>

            <br><br>
```

```
            <label for="email">Email:</label>
            <input type="email" id="email" name="email" required>

            <br><br>

            <button type="submit">Submit</button>
          </fieldset>

        </form>
    </section>
</main>

<footer>
    <p>&copy; 2026 College Symposium</p>
</footer>

</body>
</html>
```

1.3 HTTP request generated during form submission

When the student submits the form, the browser sends a POST request to /register.
For example, the HTTP request may look like:

```
POST /register HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 35

name=Rahul&email=rahul%40example.com
```

Interpretation:

- POST – sends data to the server.
- /register – destination URL.
- Host – identifies the web server.
- Content-Type – specifies the format of submitted data.
- The form data is sent in the request body.
- name and email are obtained from the name attributes of the inputs.

1.4 HTTP response cycle

The basic cycle is:
Student
↓
Fills Registration Form
↓
Clicks Submit
↓
Browser creates HTTP POST request
↓
Web Server receives request
↓
Server validates/processes data
↓
Server sends HTTP response
↓
Browser receives response
↓
Browser displays result

Example response:
HTTP/1.1 200 OK
Content-Type: text/html
<html>
<body>
<h1>Registration Successful</h1>
</body>
</html>
Common response status codes include:

- 200 OK – request successful.
- 201 Created – resource successfully created.
- 400 Bad Request – invalid request/data.
- 404 Not Found – requested resource does not exist.
- 500 Internal Server Error – server-side error.

1.5 Usability and accessibility improvements

1. Add <label> elements for every form field.
2. Use required for mandatory fields.
3. Use appropriate input types such as email.
4. Provide clear error messages.
5. Use meaningful button text such as Submit Registration.
6. Ensure sufficient color contrast.
7. Make the form keyboard accessible.
8. Use autocomplete attributes where appropriate.
9. Make the page responsive for mobile devices.
10. Use HTTPS to protect submitted information.

---

2. Cross-Browser Compatibility Issue

Given HTML

```
<html>
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

2.1 Improper nesting and missing closing tags

The following tags are not properly closed:
HTML:

```
<h1>Welcome
<p>About us
```

They should be:
HTML:

```
<h1>Welcome</h1>
<p>About us</p>
```

The document also lacks:
HTML:

```
<!DOCTYPE html>
```

and does not specify the language of the document.

2.2 How different browsers may interpret the structure

Browsers use an HTML parser that attempts to recover from malformed HTML.
For example:
HTML:

```
<h1>Welcome
<p>About us
```

may be automatically corrected internally by the browser. However, the exact DOM structure and error-recovery behavior can produce differences in rendering, spacing, styling, and scripting behavior.
Therefore, relying on browsers to automatically correct invalid HTML can result in:

- Inconsistent layouts.
- Unexpected margins and spacing.
- Different DOM structures.
- CSS behaving differently.
- JavaScript selectors producing unexpected results.

2.3 Missing semantic elements

The page should use semantic HTML5 elements such as:

- <header> – introductory content and site heading.
- <nav> – navigation links, if present.
- <main> – primary page content.
- <section> – logically grouped content.
- <footer> – footer information.

Using semantic elements improves structure, accessibility, SEO, and maintainability.

2.4 Corrected W3C-compliant HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>

<body>
<header>
  <h1>Welcome</h1>
</header>
<main>
  <section>
    <h2>About Us</h2>
    <p>About our company and its services.</p>
  </section>
</main>
<footer>
  <p>&copy; 2026 Company</p>
</footer>
</body>
</html>
```

2.5 Techniques for cross-browser compatibility

1. Use valid HTML5 and CSS.
2. Always include <!DOCTYPE html>.
3. Validate HTML and CSS using standards validators.
4. Use a CSS reset or normalization stylesheet where appropriate.
5. Avoid browser-specific or deprecated features.
6. Test on major browsers such as Chrome, Firefox, Edge, and Safari.
7. Use responsive design with appropriate media queries.
8. Use feature detection instead of assuming browser support.
9. Keep JavaScript standards-compliant.
10. Test on different screen sizes and operating systems.

3. Form Submission Failure

Given HTML:

```
<form action="/submit" method="GET">
   <input type="text" name="username">
   <input type="password" name="password">
   <button type="button">Login</button>
</form>
```

3.1 Issues in the form configuration

The main problem is:
HTML:
<button type="button">Login</button>
A button with type="button" does not submit the form by default.
It is normally used for buttons that perform custom JavaScript actions.
The form should use:
<button type="submit">Login</button>
Other improvements include:

- Add <label> elements.
- Add required attributes.
- Use HTTPS.
- Avoid transmitting passwords using plain HTTP.
- Consider using POST rather than GET for login credentials.

3.2 Why the HTTP request is not triggered

The browser submits a form when a submit action occurs.
However:
<button type="button">
is an ordinary button, not a submit button.
Therefore, clicking it does not automatically submit the form, so the browser does not generate the form's HTTP request.

Changing it to:
<button type="submit">Login</button>
causes the form submission to occur.

3.3 Role of GET method

With:
<form action="/submit" method="GET">
the form data is appended to the URL as a query string.

For example:
/submit?username=rahul&password=abc123

The corresponding request may look like:
GET /submit?username=rahul&password=abc123 HTTP/1.1
Host: example.com

Problems with GET for passwords

Using GET for login credentials is inappropriate because the URL can be stored in:

- Browser history.
- Server logs.
- Proxy logs.
- Analytics systems.
- Other URL-related records.

Therefore, passwords should not be placed in the URL.

3.4 Corrected implementation

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Login</title>
</head>

<body>

<main>
   <section>
      <h1>Login</h1>

      <form action="/submit" method="POST">

         <label for="username">Username:</label>
         <input
            type="text"
            id="username"
            name="username"
            autocomplete="username"
            required>

         <br><br>

         <label for="password">Password:</label>
         <input
            type="password"
            id="password"
            name="password"
            autocomplete="current-password"
            required>

         <br><br>
```

```
        <button type="submit">Login</button>

      </form>
    </section>
</main>

</body>
</html>
```

3.5 Improvements for secure data transmission

1. Use HTTPS/TLS rather than HTTP.
2. Use POST for submitting login credentials.
3. Never put passwords in URLs.
4. Store passwords on the server using strong password hashing, such as Argon2id or bcrypt.
5. Validate and sanitize input on the server.
6. Use secure session management after login.
7. Use Secure, HttpOnly, and appropriate SameSite cookie settings.
8. Implement protection against CSRF where applicable.
9. Do not display passwords in error messages or logs.
10. Consider rate limiting and account-lockout protections against automated login attempts.